

Simple Expense Tracker

BAUDHIK DHOLIYA

2026-09-17

- 1. Project Overview
- 2. Objectives
- 3. Technologies Used
- 4. Sample Expense Data
- 5. Basic Expense Summary
 - Total Monthly Spending
 - Average Daily Expense
 - Highest Expense
- 6. Category-Wise Expense Analysis
 - Category With Highest Spending
- 7. Bar Chart
- 8. Pie Chart
- 9. Daily Expense Chart
- 10. Budget Checking Function
- 11. Complete Project Code
- 12. Sample Output
- 13. Project Report Introduction
 - Problem Statement Objectives
 - Methodology
 - Features
 - Advantages
 - Limitations
 - Future Scope Conclusion
 -
 -
 -

1. Project Overview

The **Simple Expense Tracker** is a small R programming project used to record and analyze daily expenses.

The project allows the user to:

- Record daily expenses
- Categorize expenses such as Food, Travel, Shopping, Bills, Education and Other
- Calculate total monthly spending
- Calculate category-wise spending
- Find the highest expense
- Display expenses using bar and pie charts

2. Objectives

The main objectives of this project are:

1. To maintain daily expense records using R.
2. To organize expenses into categories.

file:///C:/Users/hardi/Downloads/Simple_Expense_Tracker.html

3. To calculate total and average spending.
4. To identify the category with the highest spending.
5. To visualize expenses using graphs.
6. To demonstrate the use of vectors, data frames, functions and basic R statistics.

3. Technologies Used

- **Programming Language:** R
- **Development Environment:** RStudio
- **Report Format:** R Markdown
- **Libraries:** ggplot2, dplyr

4. Sample Expense Data

The following sample data represents expenses for one month. The values can be replaced with your own expenses.

```
library(ggplot2)
library(dplyr)

expenses <- data.frame(
  Date = as.Date(c(
    "2026-09-01", "2026-09-02", "2026-09-03", "2026-09-05",
    "2026-09-07", "2026-09-09", "2026-09-11", "2026-09-13",
    "2026-09-15", "2026-09-16"
  )),
  Category = c(
    "Food", "Travel", "Shopping", "Food", "Education",
    "Travel", "Bills", "Food", "Shopping", "Other"
  ),
  Description = c(
    "Lunch", "Bus", "Clothes", "Dinner", "Books",
    "Auto", "Mobile Recharge", "Snacks", "Shoes", "Miscellaneous"
  ),
  Amount = c(120, 80, 750, 180, 500, 100, 299, 90, 900, 150)
)

expenses
```

```
## Date Category Description Amount ## 1 2026-09-
01 Food Lunch 120 ## 2 2026-09-02 Travel Bus 80 ##
3 2026-09-03 Shopping Clothes 750 ## 4 2026-09-05
Food Dinner 180 ## 5 2026-09-07 Education Books
500 ## 6 2026-09-09 Travel Auto 100 ## 7 2026-09-
11 Bills Mobile Recharge 299 ## 8 2026-09-13 Food
Snacks 90 ## 9 2026-09-15 Shopping Shoes 900 ## 10
2026-09-16 Other Miscellaneous 150
```

5. Basic Expense Summary

Total Monthly Spending

```
total_spending <- sum(expenses$Amount)
total_spending
```

```
## [1] 3169
```

Sample Output: The total monthly spending is ₹3,169.

Average Daily Expense

```
average_expense <- mean(expenses$Amount)
average_expense
```

```
## [1] 316.9
```

Sample Output: The average expense per recorded day is ₹316.90.

Highest Expense

```
highest_expense <- max(expenses$Amount)
highest_record <- expenses[expenses$Amount == highest_expense, ]

highest_record
```

```
##           Date Category Description Amount
## 9 2026-09-15 Shopping      Shoes      900
```

Sample Output: The highest recorded expense is ₹900 for Shopping.

6. Category-Wise Expense Analysis

The following code calculates the total amount spent in each category.

```
category_summary <- expenses %>%
  group_by(Category) %>%
  summarise(
    Total = sum(Amount),
    Number_of_Expenses = n()
  ) %>%
  arrange(desc(Total))

category_summary
```

```
## # A tibble: 6 × 3
##   Category Total Number_of_Expenses
##   <chr>      <dbl>           <int>
## 1 Shopping 1650             2
## 2 Education 500             1
## 3 Food      390             3
## 4 Bills     299             1
## 5 Travel    180             2
## 6 Other     150             1
```

Category With Highest Spending

```
highest_category <- category_summary$Category[1]
highest_category_amount <- category_summary$Total[1]

highest_category
```

```
## [1] "Shopping"
```

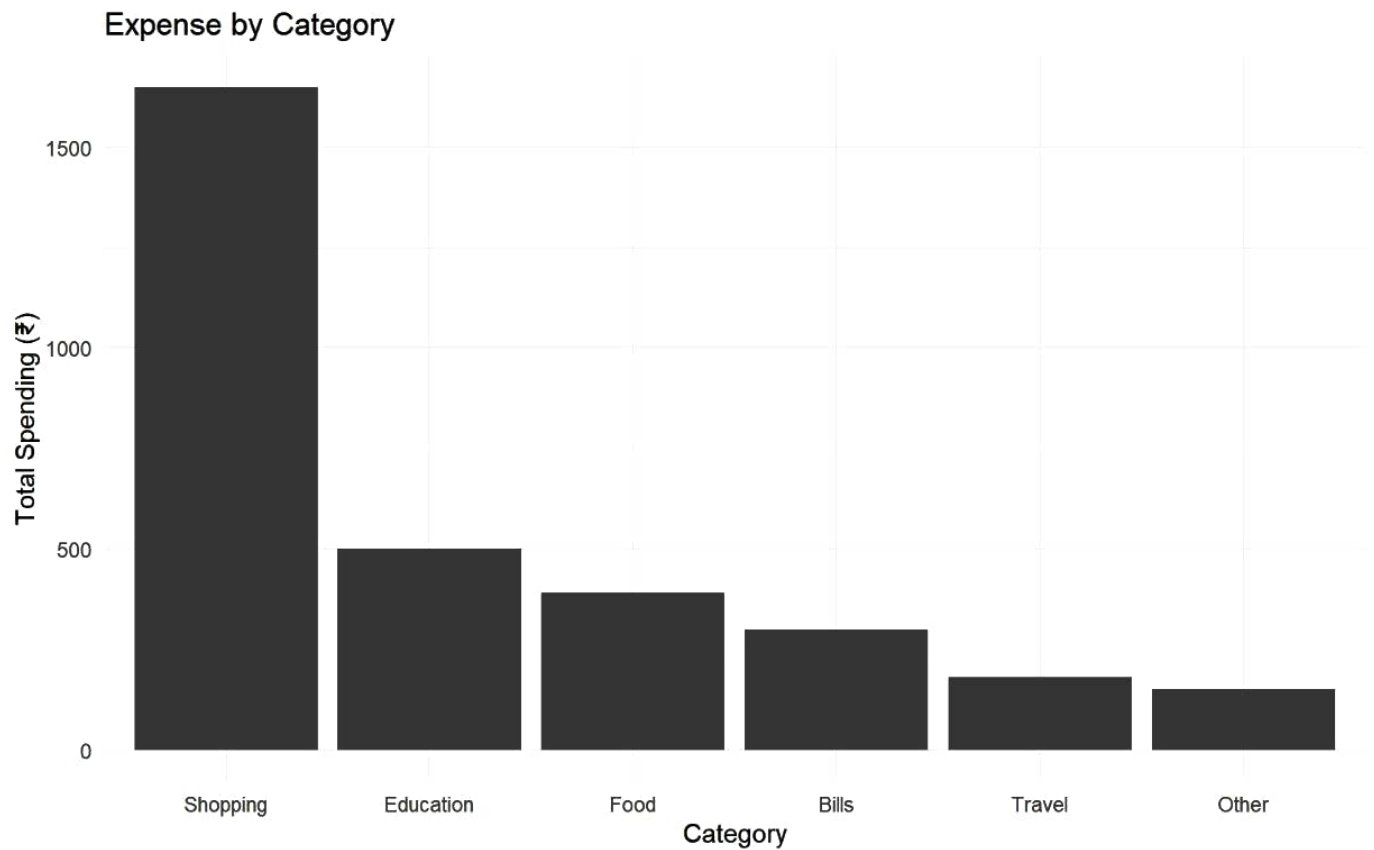
```
highest_category_amount
```

```
## [1] 1650
```

7. Bar Chart

The bar chart shows the total spending for each expense category.

```
ggplot(category_summary, aes(x = reorder(Category, -Total), y = Total)) +
  geom_bar(stat = "identity") +
  labs(
    title = "Expense by Category",
    x = "Category",
    y = "Total Spending (₹)"
  ) +
  theme_minimal()
```

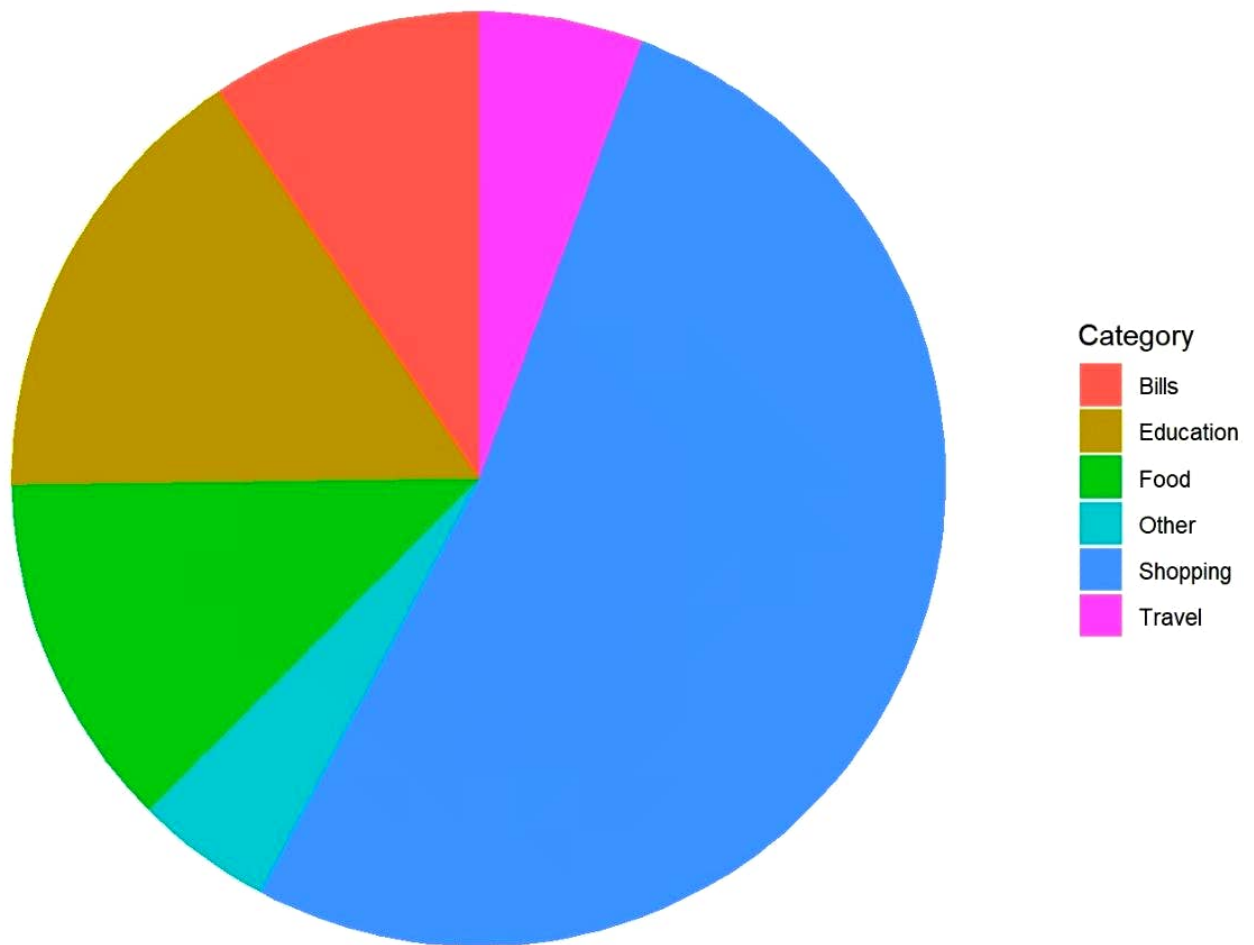


8. Pie Chart

The pie chart shows the percentage distribution of total spending.

```
ggplot(category_summary, aes(x = "", y = Total, fill = Category)) +  
  geom_col(width = 1) +  
  coord_polar("y") +  
  labs(  
    title = "Monthly Expense Distribution"  
  ) +  
  theme_void()
```

Monthly Expense Distribution

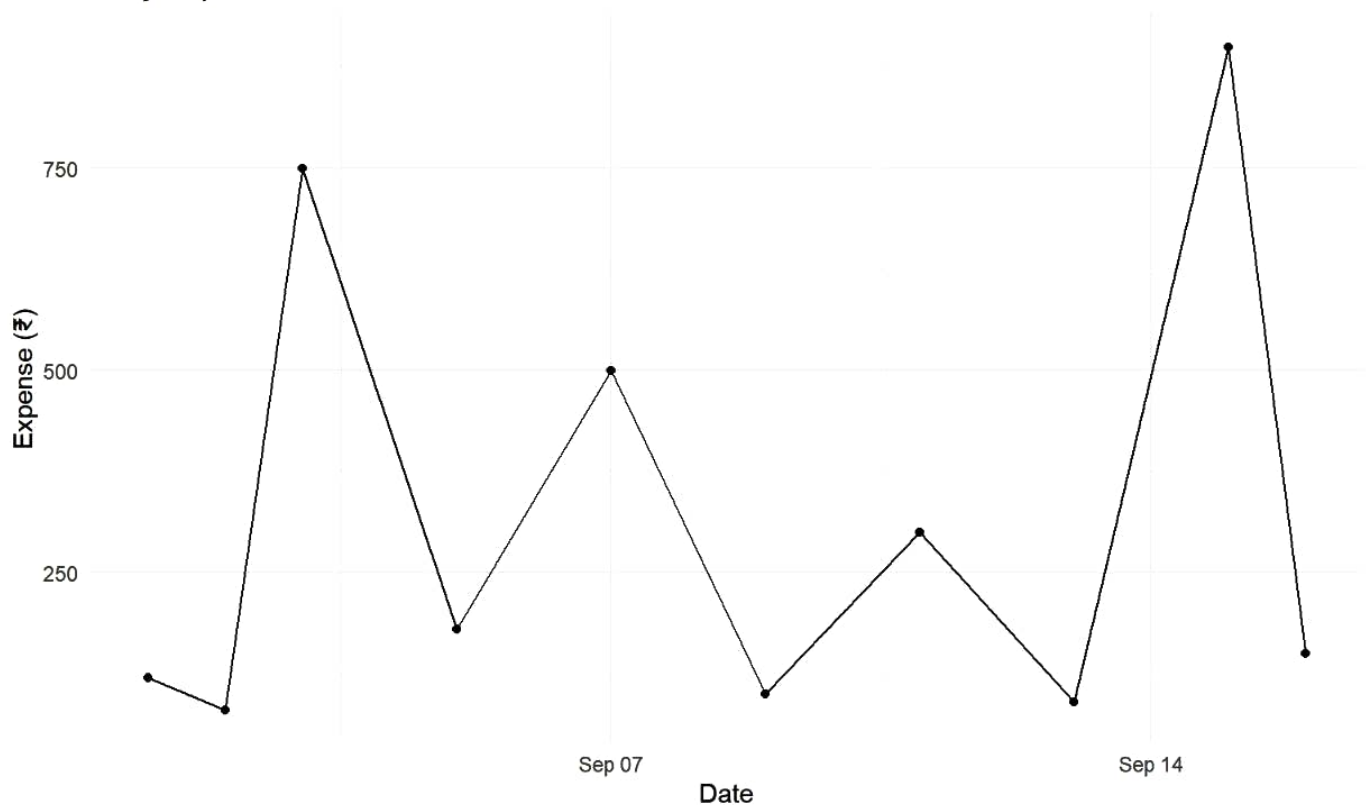


9. Daily Expense Chart

The following line chart shows the amount spent on each recorded date.

```
ggplot(expenses, aes(x = Date, y = Amount)) +  
  geom_line() +  
  geom_point() +  
  labs(  
    title = "Daily Expense Trend",  
    x = "Date",  
    y = "Expense (₹)"  
  ) +  
  theme_minimal()
```

Daily Expense Trend



10. Budget Checking Function

This function can be used to check whether the total spending is within a monthly budget.

```
check_budget <- function(budget, spending) {
  remaining <- budget - spending

  if (remaining > 0) {
    cat("Budget:", budget, "\n")
    cat("Total Spending:", spending, "\n")
    cat("Remaining Budget:", remaining, "\n")
    cat("Status: Within Budget\n")
  } else if (remaining == 0) {
    cat("Budget:", budget, "\n")
    cat("Total Spending:", spending, "\n")
    cat("Status: Budget Fully Used\n")
  } else {
    cat("Budget:", budget, "\n")
    cat("Total Spending:", spending, "\n")
    cat("Amount Over Budget:", abs(remaining), "\n")
    cat("Status: Budget Exceeded\n")
  }
}
```

```
check_budget(4000, total_spending)
```

```
## Budget: 4000
## Total Spending: 3169
## Remaining Budget: 831
## Status: Within Budget
```

11. Complete Project Code

The following is the complete executable R code in one block.

```

library(ggplot2)
library(dplyr)

expenses <- data.frame(
  Date = as.Date(c(
    "2026-09-01", "2026-09-02", "2026-09-03", "2026-09-05",
    "2026-09-07", "2026-09-09", "2026-09-11", "2026-09-13",
    "2026-09-15", "2026-09-16"
  )),
  Category = c(
    "Food", "Travel", "Shopping", "Food", "Education",
    "Travel", "Bills", "Food", "Shopping", "Other"
  ),
  Description = c(
    "Lunch", "Bus", "Clothes", "Dinner", "Books",
    "Auto", "Mobile Recharge", "Snacks", "Shoes", "Miscellaneous"
  ),
  Amount = c(120, 80, 750, 180, 500, 100, 299, 90, 900, 150)
)

cat("----- SIMPLE EXPENSE TRACKER -----\\n\\n")

total_spending <- sum(expenses$Amount)
average_expense <- mean(expenses$Amount)

cat("Total Monthly Spending: ₹", total_spending, "\\n", sep = "")
cat("Average Recorded Expense: ₹", round(average_expense, 2), "\\n\\n", sep = "")

highest_expense <- max(expenses$Amount)
highest_record <- expenses[expenses$Amount == highest_expense, ]

cat("Highest Expense: ₹", highest_expense, "\\n", sep = "")
cat("Category: ", highest_record$Category, "\\n", sep = "")
cat("Description: ", highest_record$Description, "\\n\\n", sep = "")

category_summary <- expenses %>%
  group_by(Category) %>%
  summarise(
    Total = sum(Amount),
    Number_of_Expenses = n()
  ) %>%
  arrange(desc(Total))

print(category_summary)

check_budget <- function(budget, spending) {
  remaining <- budget - spending

  if (remaining > 0) {
    cat("\\nWithin Budget\\n")
    cat("Remaining Budget: ₹", remaining, "\\n", sep = "")
  } else if (remaining == 0) {
    cat("\\nBudget Fully Used\\n")
  } else {
    cat("\\nBudget Exceeded\\n")
  }
}

```

```

    cat("Amount Over Budget: ₹", abs(remaining), "\n", sep = "")
  }
}

check_budget(4000, total_spending)

ggplot(category_summary, aes(x = reorder(Category, -Total), y = Total)) +
  geom_bar(stat = "identity") +
  labs(
    title = "Expense by Category",
    x = "Category",
    y = "Total Spending (₹)"
  ) +
  theme_minimal()

ggplot(category_summary, aes(x = "", y = Total, fill = Category)) +
  geom_col(width = 1) +
  coord_polar("y") +
  labs(title = "Monthly Expense Distribution") +
  theme_void()

```

12. Sample Output

----- SIMPLE EXPENSE TRACKER -----

Total Monthly Spending: ₹3169

Average Recorded Expense: ₹316.90

Highest Expense: ₹900

Category: Shopping

Description: Shoes

Category-wise Summary:

Category	Total	Number of Expenses
Shopping	1650	2
Food	390	3
Education	500	1
Bills	299	1
Travel	180	2
Other	150	1

Budget: ₹4000

Total Spending: ₹3169

Remaining Budget: ₹831

Status: Within Budget

13. Project Report

Introduction

Managing daily expenses is important for maintaining a personal budget. The **Simple Expense Tracker** is an R-based mini project that records daily expenses and analyzes them according to different categories.

Problem Statement

People often spend money on different activities such as food, transportation, shopping, education and bills. Without proper records, it can be difficult to determine where the money is being spent. This project provides a simple method for recording and analyzing expenses.

Objectives

- Record daily expenses.
- Categorize expenses.
- Calculate total monthly spending.
- Calculate average expense.
- Find the highest expense.
- Analyze category-wise spending.
- Check spending against a budget.
- Visualize expenses using charts.

Methodology

The project uses an R data frame to store expense information. Each record contains the date, category, description and amount. R functions such as `sum()`, `mean()` and `max()` are used for calculations.

The `dplyr` package is used for category-wise grouping and summarization. The `ggplot2` package is used to create bar, pie and line charts.

Features

1. Daily expense recording
2. Expense categorization
3. Total spending calculation
4. Average expense calculation
5. Highest expense identification
6. Category-wise analysis
7. Budget checking
8. Graphical visualization

Advantages

- Simple and easy to use.
- Helps monitor spending.
- Provides quick financial summaries.
- Graphs make the data easier to understand.
- Can be modified for personal use.

Limitations

- The sample version stores data only during the R session.
- It does not have a login system.
- It does not connect to a bank account.
- Expenses must be entered manually.

Future Scope

The project can be improved by adding:

- A graphical user interface using Shiny.
- CSV/Excel file storage.
- Monthly and yearly reports.
- Automatic budget alerts.
- More detailed financial charts.
- Expense editing and deletion.
- Search and filter functionality.

Conclusion

The Simple Expense Tracker demonstrates how R can be used for basic data management, calculations and visualization. It combines data frames, functions, conditional statements, data manipulation and graphical representation into a practical mini project.